

[illegible]

```

9      float64
10     float64
11     float64
12     float64
13     float64
14      int64
dtype: object

```

	1	2	3	4	5
6 \					
count	270.000000	270.000000	270.000000	270.000000	270.000000
mean	54.433333	0.677778	3.174074	131.344444	249.659259
std	9.109067	0.468195	0.950090	17.861608	51.686237
min	29.000000	0.000000	1.000000	94.000000	126.000000
25%	48.000000	0.000000	3.000000	120.000000	213.000000
50%	55.000000	1.000000	3.000000	130.000000	245.000000
75%	61.000000	1.000000	4.000000	140.000000	280.000000
max	77.000000	1.000000	4.000000	200.000000	564.000000

	7	8	9	10	11
12 \					
count	270.000000	270.000000	270.000000	270.000000	270.000000
mean	1.022222	149.677778	0.329630	1.050000	1.585185
std	0.997891	23.165717	0.470952	1.145210	0.614390
min	0.000000	71.000000	0.000000	0.000000	1.000000
25%	0.000000	133.000000	0.000000	0.000000	1.000000
50%	2.000000	153.500000	0.000000	0.800000	2.000000
75%	2.000000	166.000000	1.000000	1.600000	2.000000
max	2.000000	202.000000	1.000000	6.200000	3.000000

	13	14
count	270.000000	270.000000
mean	4.696296	1.444444
std	1.940659	0.497827

min	3.000000	1.000000
25%	3.000000	1.000000
50%	3.000000	1.000000
75%	7.000000	2.000000
max	7.000000	2.000000

Q2. Prepare the data for Logistic Regression

```
X = df.iloc[:, :-1] # All columns except the last one
y = df.iloc[:, -1] # The last column as target
# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Logistic Regression without SGD

```
lr = LogisticRegression(max_iter=1000, C=1.0)
lr.fit(X_train, y_train)

LogisticRegression(max_iter=1000)
```

Q3. Confusion Matrix

```
y_pred = lr.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix (without SGD):\n", cm)

Confusion Matrix (without SGD):
[[32  1]
 [ 3 18]]
```

Q4. Accuracy, Precision, F-measure, Recall

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f_measure = f1_score(y_test, y_pred)
print("Without SGD - Accuracy: {}, Precision: {}, Recall: {}, F-
measure: {}".format(accuracy, precision, recall, f_measure))

Without SGD - Accuracy: 0.9259259259259259, Precision:
0.9142857142857143, Recall: 0.9696969696969697, F-measure:
0.9411764705882354
```

Q5. Logistic Regression with SGD

```
sgd = SGDClassifier(loss='log_loss', alpha=0.0001, max_iter=1000,  
tol=1e-3, random_state=42)  
sgd.fit(X_train, y_train)
```

```
SGDClassifier(loss='log_loss', random_state=42)
```

Q6. Confusion Matrix with SGD

```
y_pred_sgd = sgd.predict(X_test)  
cm_sgd = confusion_matrix(y_test, y_pred_sgd)  
print("Confusion Matrix (with SGD):\n", cm_sgd)
```

```
Confusion Matrix (with SGD):  
[[28  5]  
 [ 5 16]]
```

Q7. Accuracy, Precision, F-measure, Recall with SGD

```
accuracy_sgd = accuracy_score(y_test, y_pred_sgd)  
precision_sgd = precision_score(y_test, y_pred_sgd)  
recall_sgd = recall_score(y_test, y_pred_sgd)  
f_measure_sgd = f1_score(y_test, y_pred_sgd)  
print("With SGD - Accuracy: {}, Precision: {}, Recall: {}, F-measure:  
{}, format(accuracy_sgd, precision_sgd, recall_sgd, f_measure_sgd))
```

```
With SGD - Accuracy: 0.8148148148148148, Precision:  
0.8484848484848485, Recall: 0.8484848484848485, F-measure:  
0.8484848484848486
```

Q8. Plotting the results

```
labels = ['Accuracy', 'Precision', 'Recall', 'F-measure']  
without_sgd = [accuracy, precision, recall, f_measure]  
with_sgd = [accuracy_sgd, precision_sgd, recall_sgd, f_measure_sgd]  
x = range(len(labels))  
plt.plot(x, without_sgd, label='Without SGD', marker='o')  
plt.plot(x, with_sgd, label='With SGD', marker='o')  
plt.xticks(x, labels)  
plt.ylabel('Score')  
plt.title('Comparison of Logistic Regression and SGD')
```

```
plt.legend()  
plt.show()
```

